讲解 1 / 3：先区分保存路径改造与规划后端迁移

原始版：v1 同步基线

本报告的对照实现

规划：MotionGen
环境：8 环境生成配置
保存：主流程同步导出

压缩与写入完成后
生成控制流才能继续

已使用 GPU 与多环境
不是“整个系统完全串行”

改进版：v1 异步保存

主要改变保存侧

规划：保留 v1 后端
环境：保留 8 环境
保存：独立 CPU writer

共享内存移交数据
分片保存 → 排空 → 合并

生成—保存流水线重叠
另增加保存侧任务并行

cuRobo v2 适配版

主要改变规划侧

规划：MotionPlanner
环境：保留 8 环境
保存：沿用异步保存池

适配模型与目标表示
插值轨迹 → 关节重排 → FK

规划后端迁移与接口适配
不是新增 GPU 生产进程

当前依据：v1 同步基线、冻结异步保存实现及 v2 规划适配器。

支持结论：保存路径改造与规划后端迁移是两项变化；三版均保留 8 环境生成配置。

讲解 2 / 3：沿调用链说明具体改动

原始版：同步保存



同步导出占用生成控制流。

改进版：异步保存



Bridge.export → ExportPool.submit；提交后生成可继续，超预算仍会等待。

v2 版：规划适配 + 异步保存



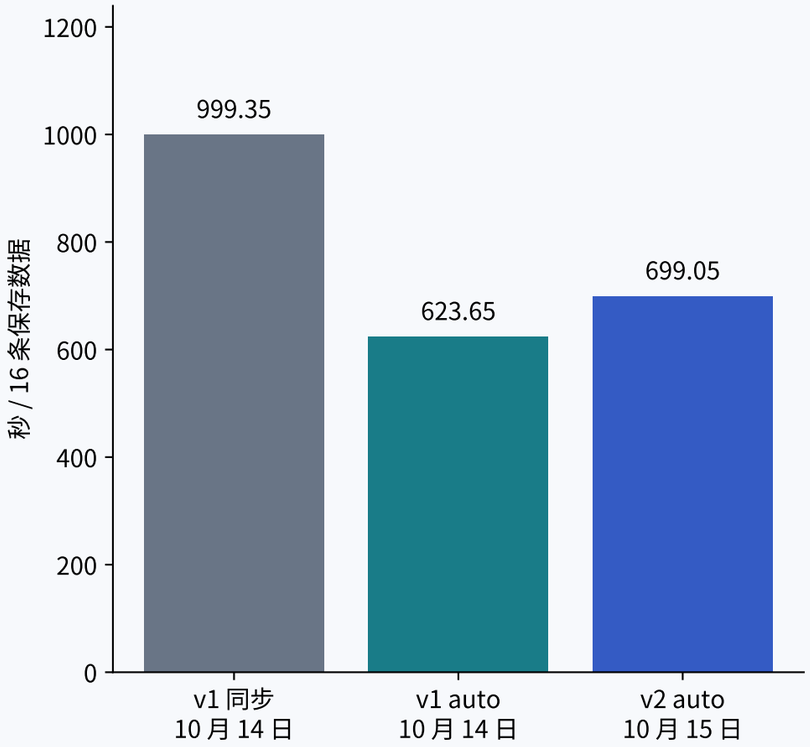
规划路径在前端变化；episode 保存仍走共享内存与 writer 池。

当前依据：recorder、Bridge.export / ExportPool.submit 与 v2 plan_pose 的调用链。

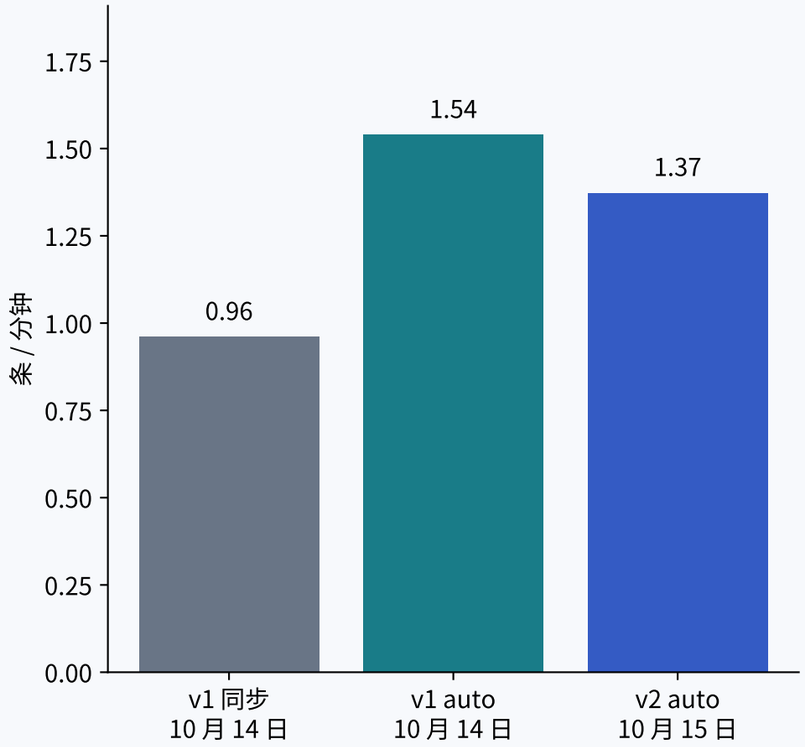
支持结论：异步版转移保存工作；v2 版适配规划输入输出，示意图不表示实测时序。

讲解 3 / 3：实现改变不等于性能优势，需结合完整输出与质量

最终就绪时间 ↓



记录成功吞吐 ↑



解释与质量边界

v1 auto 对比同步
耗时减少 37.6%
吞吐提高 60.2%
末态均通过 16/16

v2 auto：末态通过 15/16
完成接口迁移与整轮运行
尚未建立性能优势

当前实验：v1 同步与 auto 为同日单轮；v2 auto 为次日重跑，均保存 16 条。

支持结论：v1 auto 本轮效率提高；v2 末态仅 15/16 通过，尚不能确立其性能优势。

流程分段：并行化发生在哪里，哪里仍需等待

阶段	执行方式	同步点与边界
任务调度	asyncio 协程并发	共享主线程；等待未完成时挂起
规划计算	GPU 算子内部数据并行	主机逐环境调用；跨请求重叠未测
仿真步进	动作汇合后批量 env.step	等待整批动作；不是每环境一个进程
录制 / 内存移交	主流程 clone、pack、submit	CPU 帧所有权调整；仍有打包与预算等待
压缩 / 写入	spawn CPU 进程处理不同 episode	独立 HDF5 分片；共享 CPU 与磁盘资源
回执 / 收尾	主流程 poll、finish、merge	回执后回收内存；排空后合并最终输出

当前依据：环境主循环、规划调用、录制包装与冻结保存池源码。

支持结论：各阶段采用不同并发机制；规划跨请求重叠与渲染并发尚无设备追踪证明。

接口迁移：目标类型、参数顺序、配置传递和轨迹语义均需适配

v1：当前适配器调用

配置构造
MotionGenConfig.load_from_robot_config(...)
MotionGenPlanConfig(...)

目标与调用
Pose
plan_single(start_state, goal_pose, plan_config)

轨迹结果
优先 optimized_plan，空时取插值
get_full_js → get_ordered_joint_state

v2：实验冻结适配器调用

配置构造
MotionPlannerCfg.create(...)
robot / scene_model / device_cfg

目标与调用
GoalToolPose.from_poses(...)
plan_pose(goals, start_state, max_attempts=...)

轨迹结果
get_interpolated_plan()
reorder(joint_names) → FK → 世界系位姿

上层保留 update_world_and_plan_motion(...) → bool；enable_retiming 默认值仍有差异，详见正文。

当前依据：本地 v1 适配器与实验冻结 v2_planner.py，详细行号见接口证据。

支持结论：迁移涉及配置、目标、调用与轨迹语义；保持上层任务契约不等于库 API 相同。

cuRobo 库级变化 ≠ 本次 AutoData 全部启用并验证

维度	库级能力变化	本次实验覆盖
GPU 并行	v1 已有多初值并行求解 v2 继续扩展 GPU 计算	每环境规划器；未实现跨环境批量请求 不把 CPU writer 当作 GPU 并行升级
轨迹优化	v1 已有平滑性目标 v2 强调 B-spline 与动力学约束	读取 v2 插值轨迹 未验证本任务力矩约束激活与满足
场景感知	v2 增强 GPU 原生 深度融合 TSDF / ESDF	从 USD 解析碰撞几何 没有深度融合建图对照
高自由度计算	v2 强调拓扑感知运动学 可微逆动力学与自碰撞计算	OpenArm 左臂任务 没有全身机器人性能对照
性能结论	官方结果对应其测试任务 不能直接转成生产吞吐收益	v2 完整保存 16 条、末态通过 15/16 跨日期单轮，尚未确立性能优势

当前依据：官方 v1 文档、v2 仓库与论文，以及本项目冻结适配代码。

支持结论：库具备的能力不等于本实验已启用验证；不引用库基准推算本项目提速。

00 代码差别在导出路径和扩容决策，共用 cuRobo v1 生成任务

同步 · workers=0	固定上限 · 4 / 8	auto
保留原 recorder 导出	ExportPool._scale_workers	ResourceGovernor.decide
episode 完成 ↓ 主进程压缩 + 写 HDF5 ↓ 继续生成	pending 有积压 ↓ desired = min(pending, cap) ↓ 按需启动 CPU writer	pending 有积压 ↓ CPU / 内存 / I/O 等准入 ↓ 通过：增加 1 个 writer
主流程承担保存等待	本轮两种上限均启动 4 个	至少间隔 1 秒；不主动缩容

当前依据：v1 实验冻结源码；同步、固定上限与 auto 共用生成任务。

支持结论：实现差别在保存路径与扩容策略；没有单项优化的消融结果。

15 保存机制的串并行对照：所有生产配置仍使用 8 个仿真环境

同步保存 · workers=0



同一主流程承担保存等待

异步单 writer



提交后可继续生成；保存侧仍只有一个执行者

异步多 writer / auto



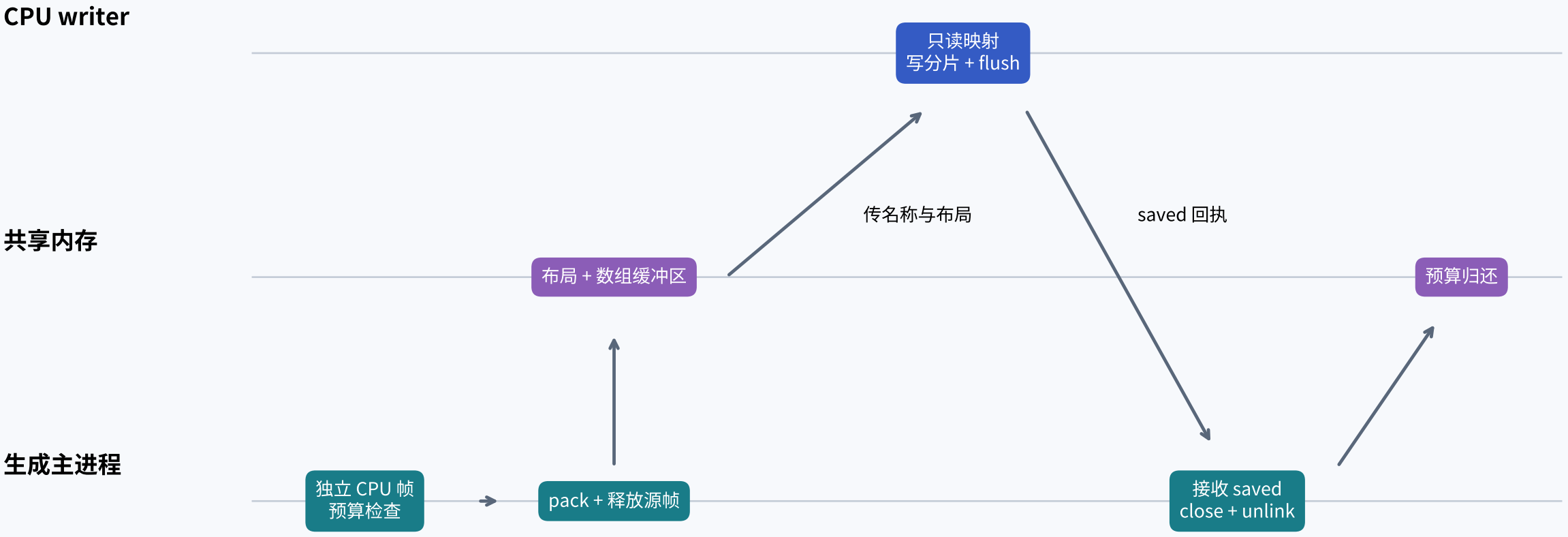
生成与保存重叠；在途内存超预算时等待，最终仍需合并

控制流示意，无耗时比例。异步提交后生成可继续，后台保存同时推进；不是三个版本的实测时间线。

当前依据：同步、单 writer、多 writer 的保存流程；所有生产配置均为 8 环境。

支持结论：流水线重叠与保存侧多进程是两种机制；图中没有实测时间比例。

12 数据所有权与回收：必须等 writer 确认后才能释放



示意图，无时间比例；队列预算不足会等待回执，writer 写入可与下一条生成重叠。

当前依据：冻结源码中的共享内存打包、saved 回执和释放流程。

支持结论：回执后释放数据并归还预算；未单独测量该机制的提速贡献。

13 cuRobo v2：保留 AutoData 任务契约，适配模型、规划与轨迹接口

输入适配

原机器人配置深拷贝
ee_link → tool_frames
retract_config →
default_joint_position

目标 / 物体 → 基座系
开闭夹爪参数原地更新

规划与接触

MotionPlanner.plan_pose
GoalToolPose 目标

搬运时附着物体
接触阶段临时调整碰撞
finally 恢复 / detach

输出适配

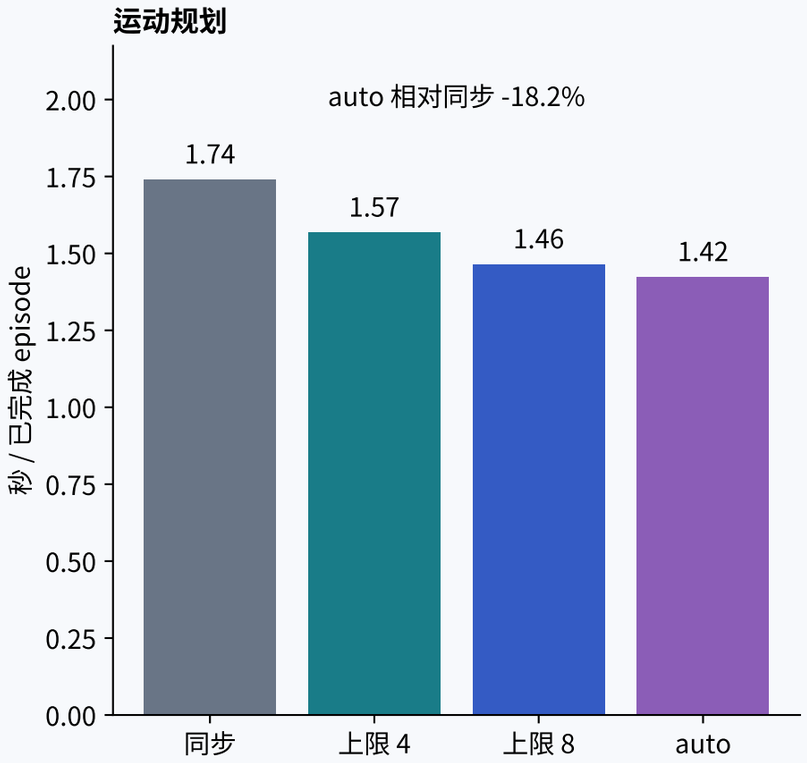
get_interpolated_plan
读取稠密执行轨迹

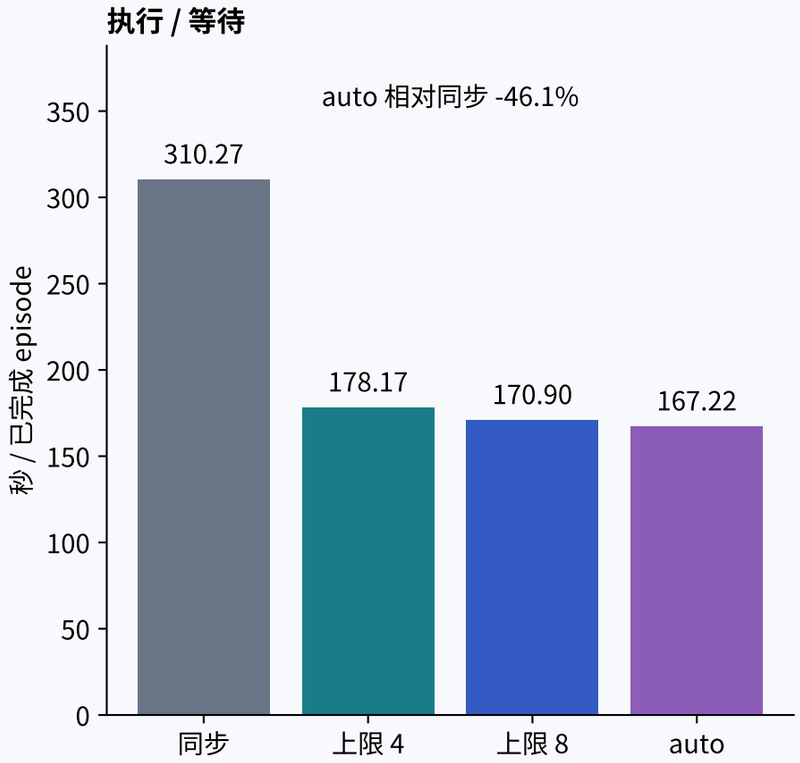
按 joint_names 重排
可选关节距离重采样
FK → 世界系末端位姿

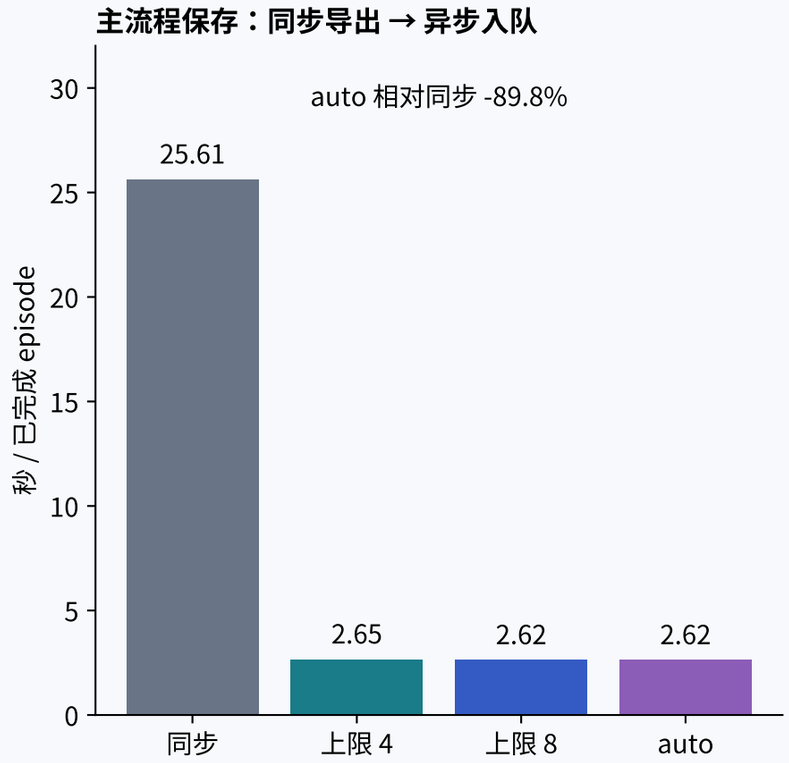
8 个环境各有规划对象；不把样条控制点直接执行，也不把规划成功等同于最终数据合格。

当前依据：v2 适配器的模型配置、规划调用与轨迹输出接口。

支持结论：v2 已接入生产流程；接口适配本身不能证明速度或质量优于 v1。



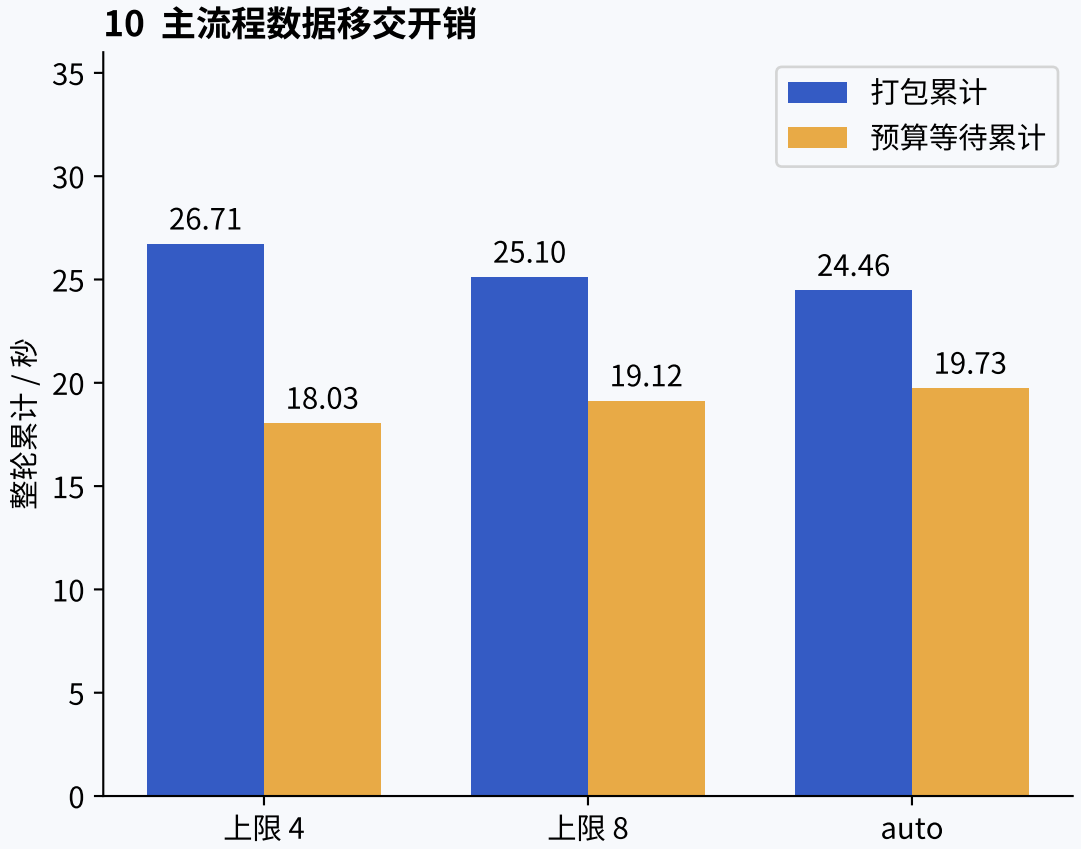


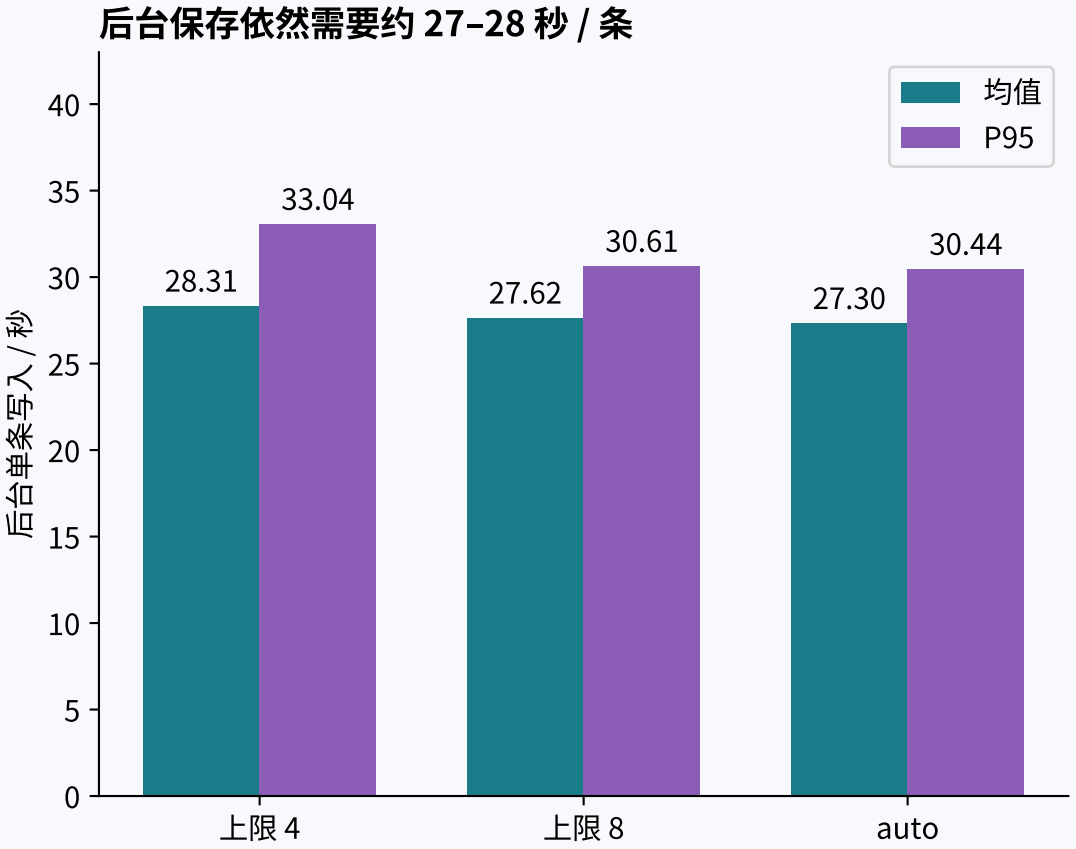


各组 17 个完成样本（含 1 个失败）；右图两端工作不同，反映主流程负担而非后台写入加速。

当前实验：v1 四组各 17 个完成 episode 的规划、执行／等待及导出计时。

支持结论：主流程负担降低；同步导出与异步入队工作范围不同，不是写盘速度比。



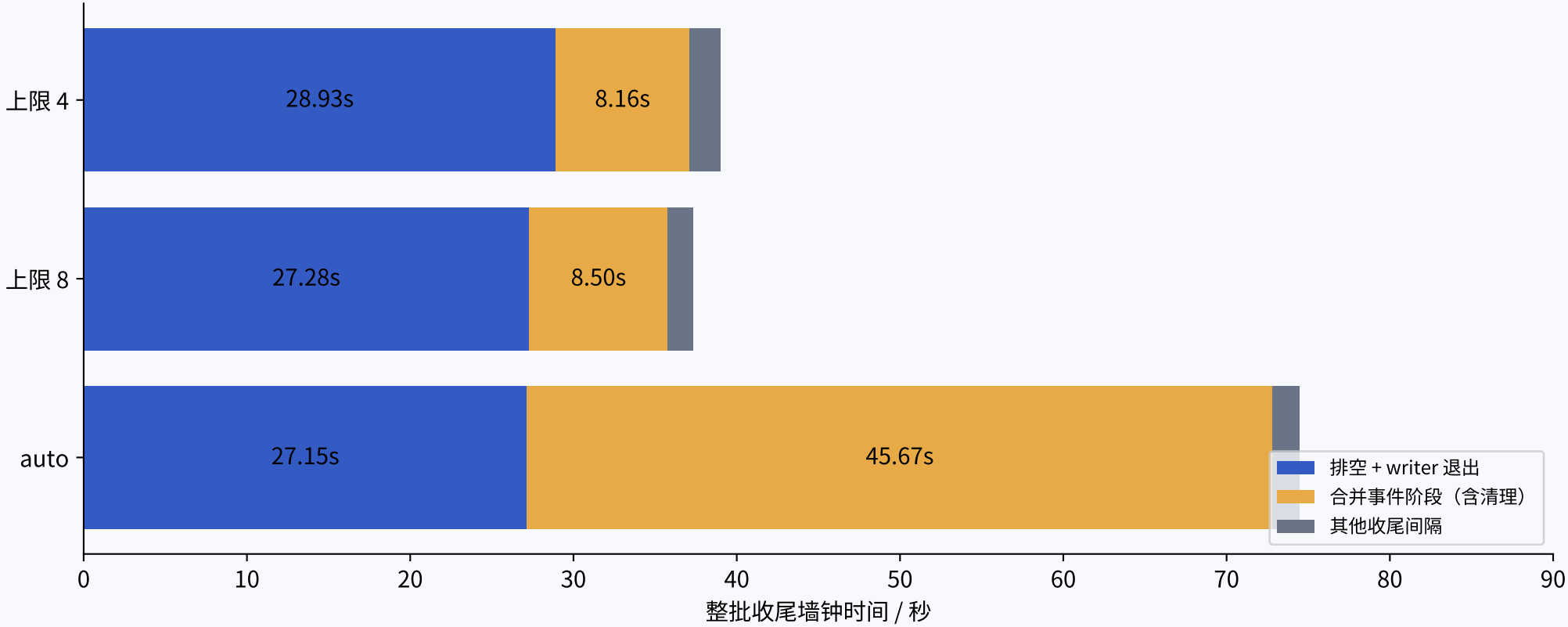


右图每组 16 个保存事件；后台并发耗时不能与左图相加为墙钟时间。

当前实验：异步三组打包与预算等待累计值，以及每组 16 次后台保存计时。

支持结论：数据移交仍有开销，后台每条保存约 27–28 秒；并发耗时不可直接相加。

11 auto 的合并阶段抵消了生成区间的领先

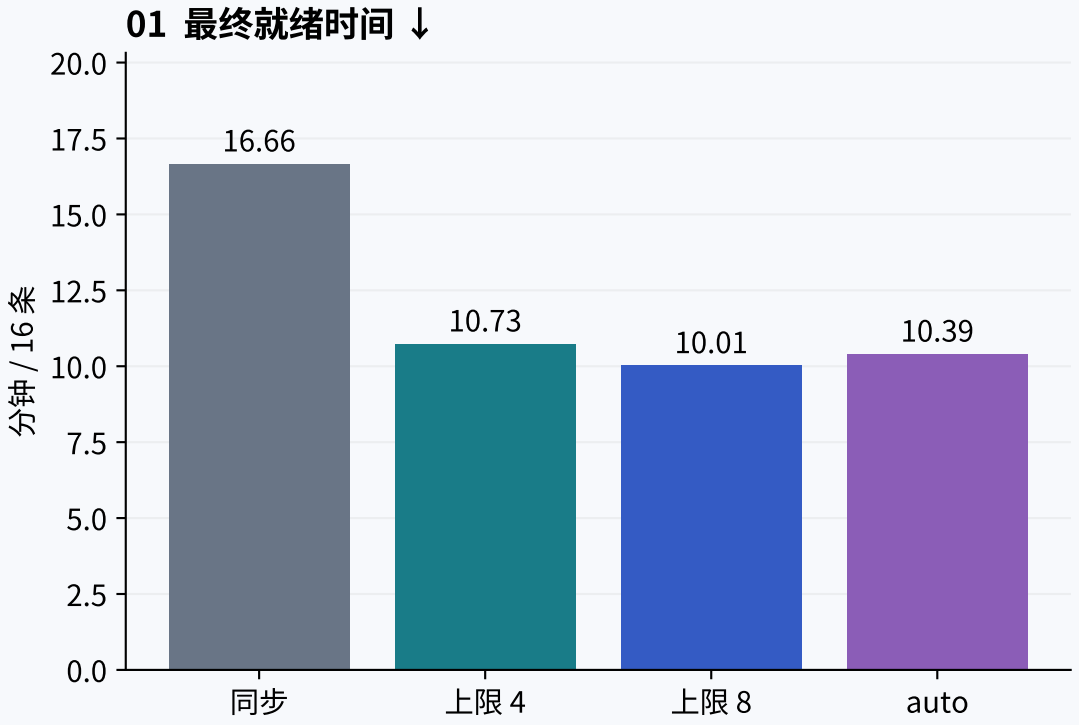


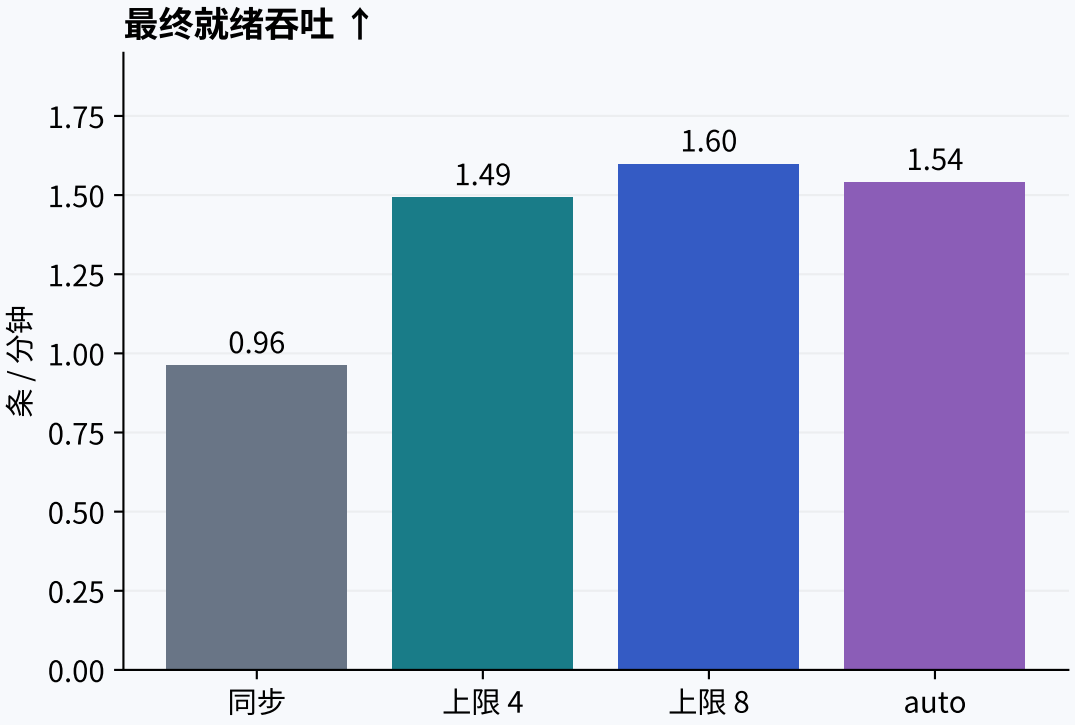
区间：最后 episode 完成 → output_ready。其他间隔为差值，并非独立埋点；同步收尾仅 1.68 秒。

当前实验：异步三组从最后 episode 完成到 output_ready 的收尾计时。

支持结论：本轮 auto 的合并事件阶段较长；单次结果不能证明 auto 必然更慢。

8 环境 · seed 101 · cuRobo v1 · 单轮观测

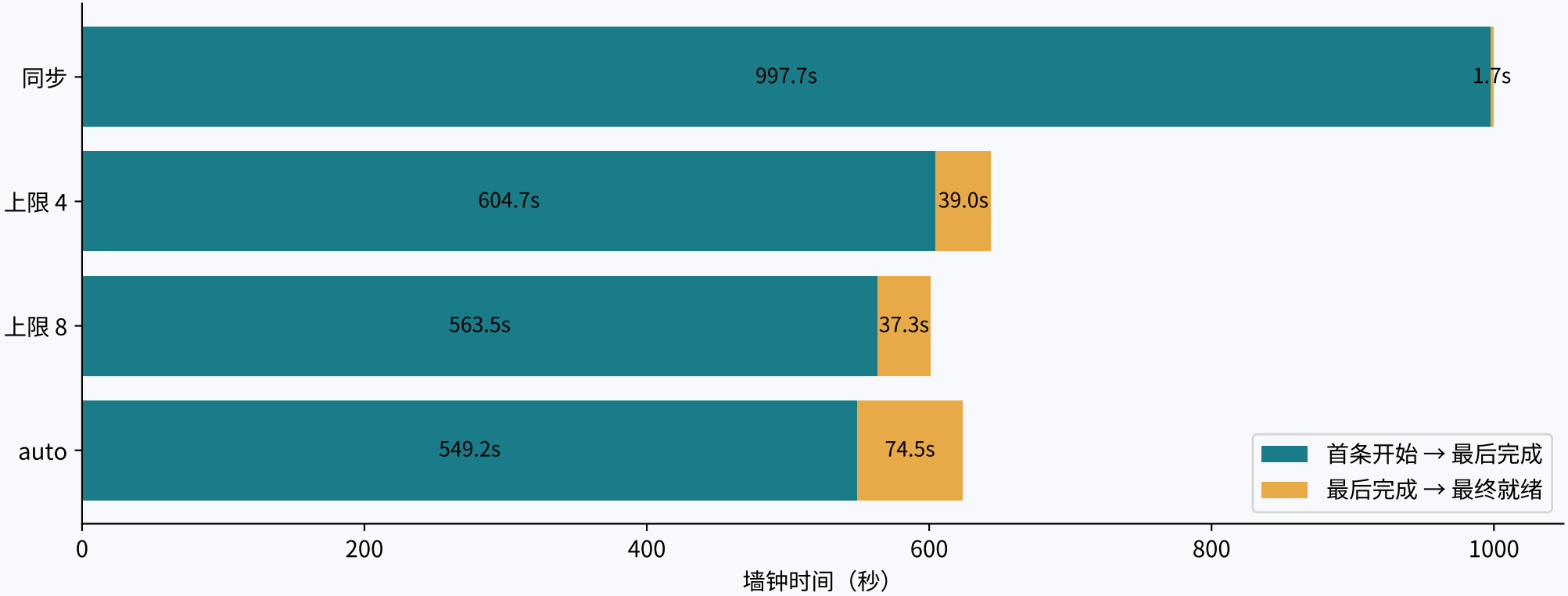




当前实验：v1，8 环境、seed 101，每组最终保存 16 条，单轮比较。

支持结论：异步组吞吐提高约 55%–66%；不能据单轮结果确定稳定排名。

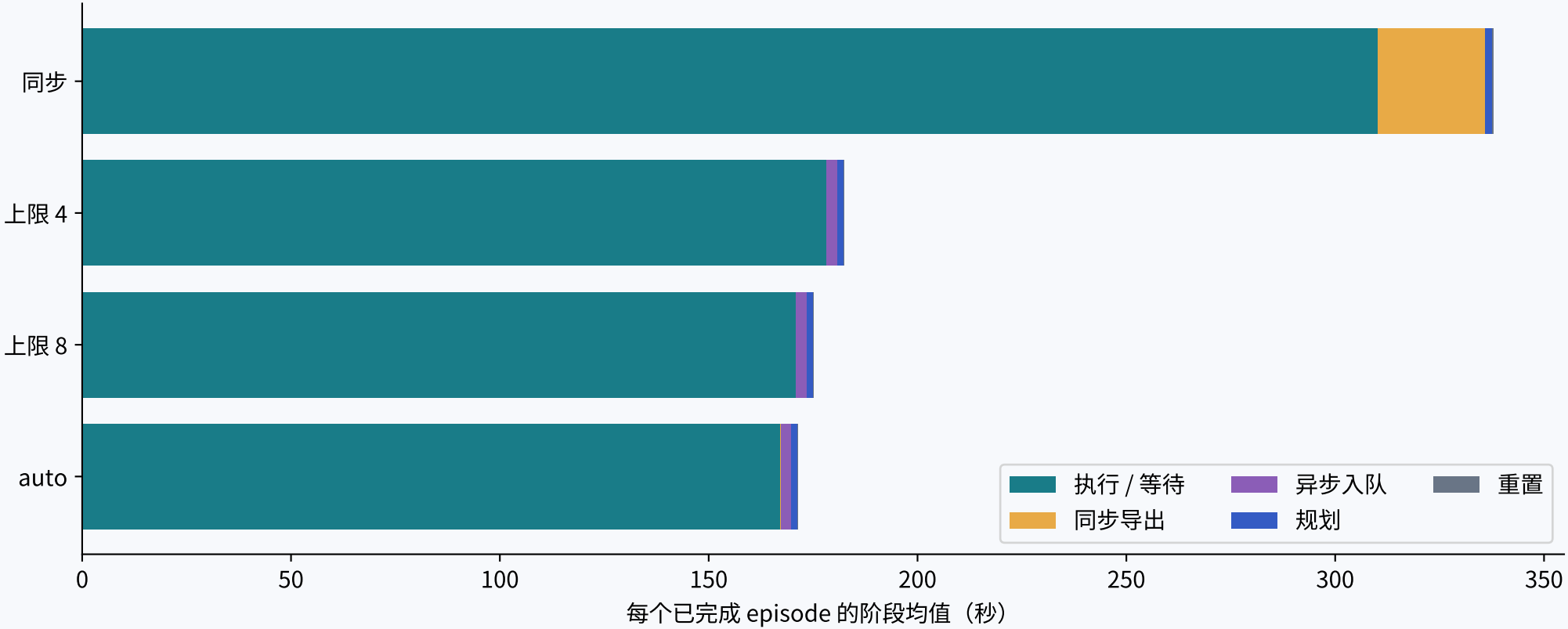
02 生成结束不等于数据可用



当前实验：将首条开始至最终就绪拆分为生成区间与收尾区间。

支持结论：最后一条生成完成后仍需收尾；生成结束不能代替文件就绪。

03 阶段耗时：执行等待和同步导出下降

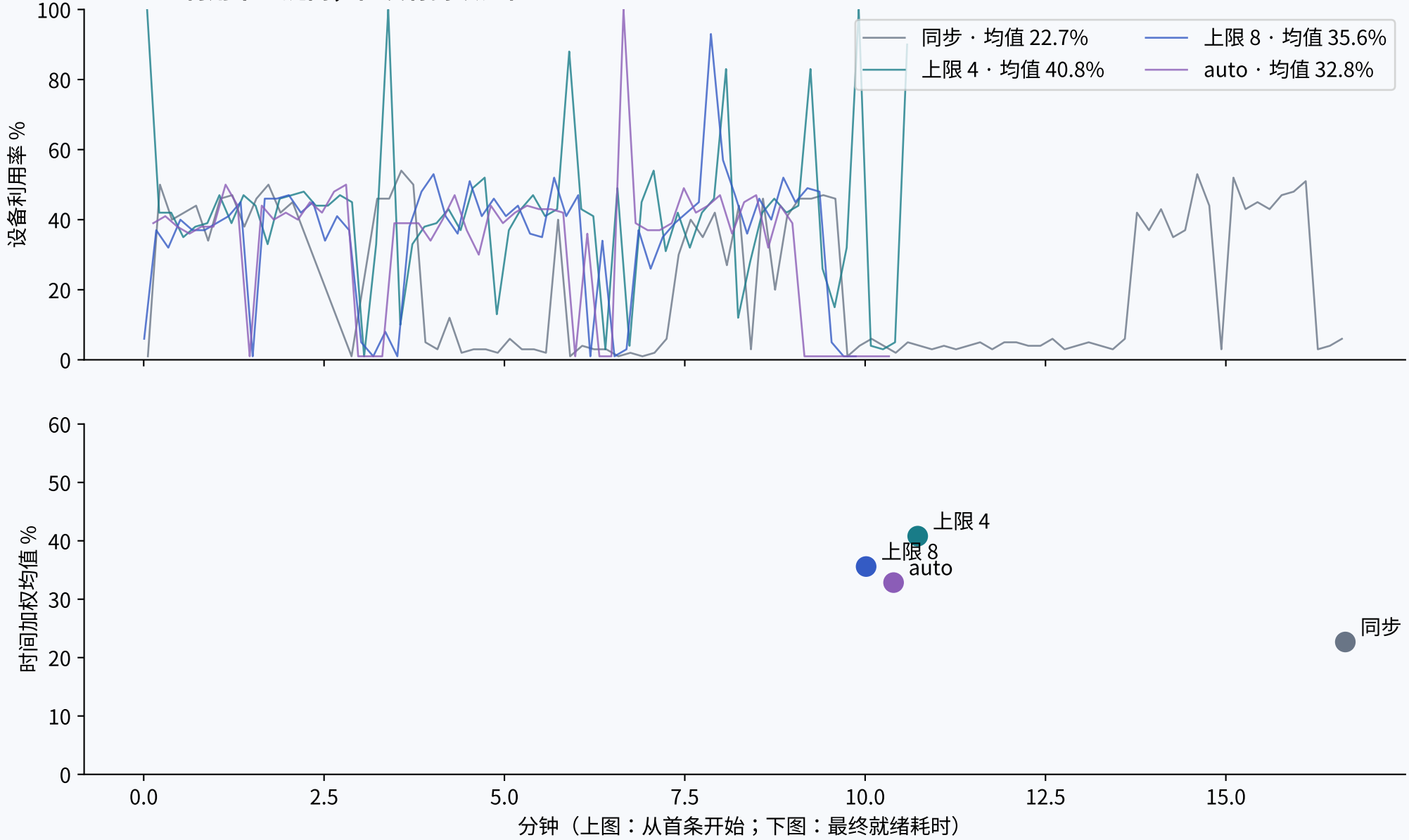


并发 episode 的耗时会重叠；此图不等于整批墙钟时间，也不覆盖所有未计时开销。

当前实验：各组 17 个完成 episode（含 1 次失败）的阶段均值。

支持结论：执行／等待和主流程导出负担下降；阶段均值不能相加为整批耗时。

04 GPU 利用率：提高，但没有持续饱和

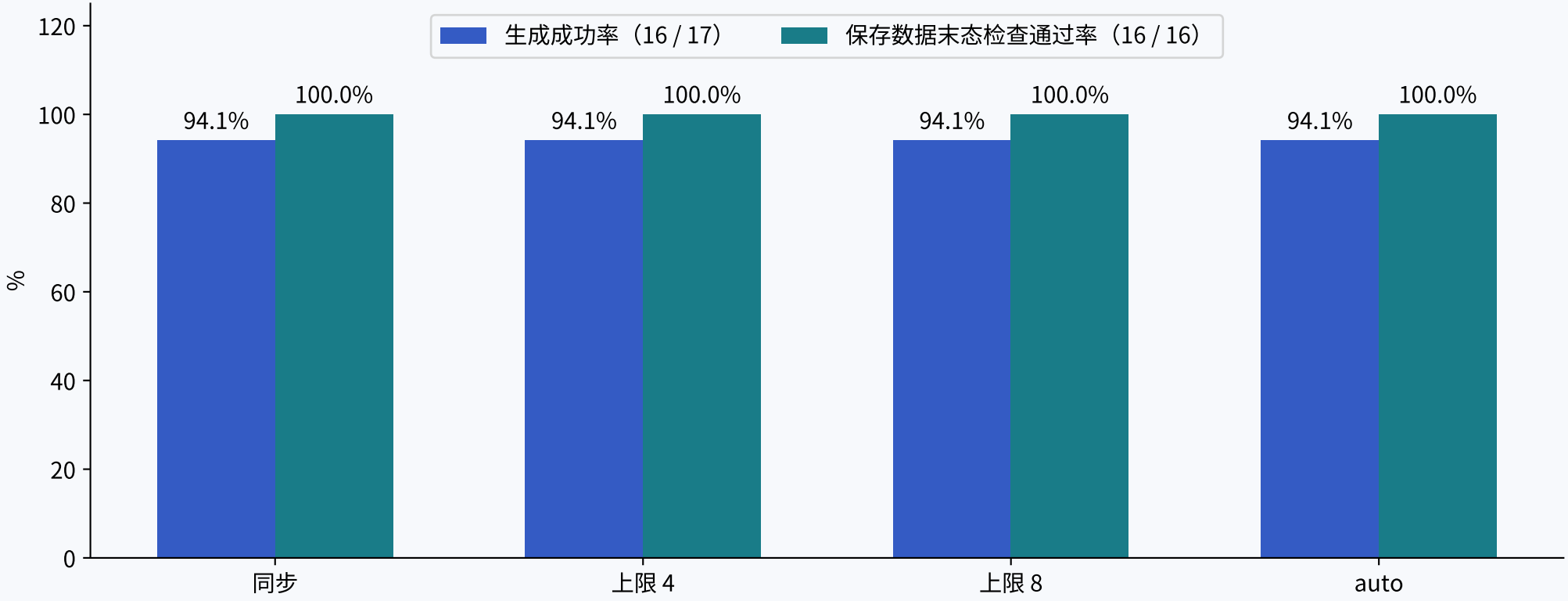


均值采用首条开始至 output_ready 内有效采样的左保持时间加权；设备级指标包含其他进程。

当前实验：v1 四组运行期间的设备级 GPU 利用率采样。

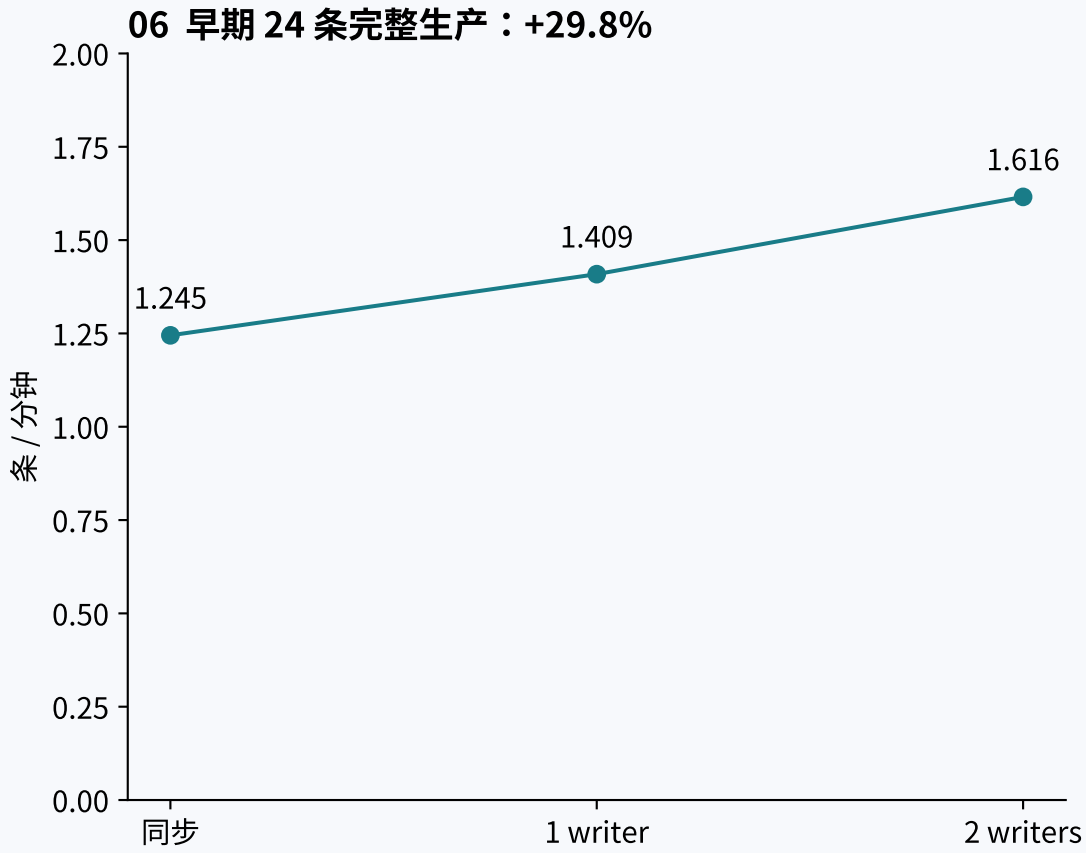
支持结论：异步组平均利用率较高；不能证明 kernel 重叠或 GPU 算法提速。

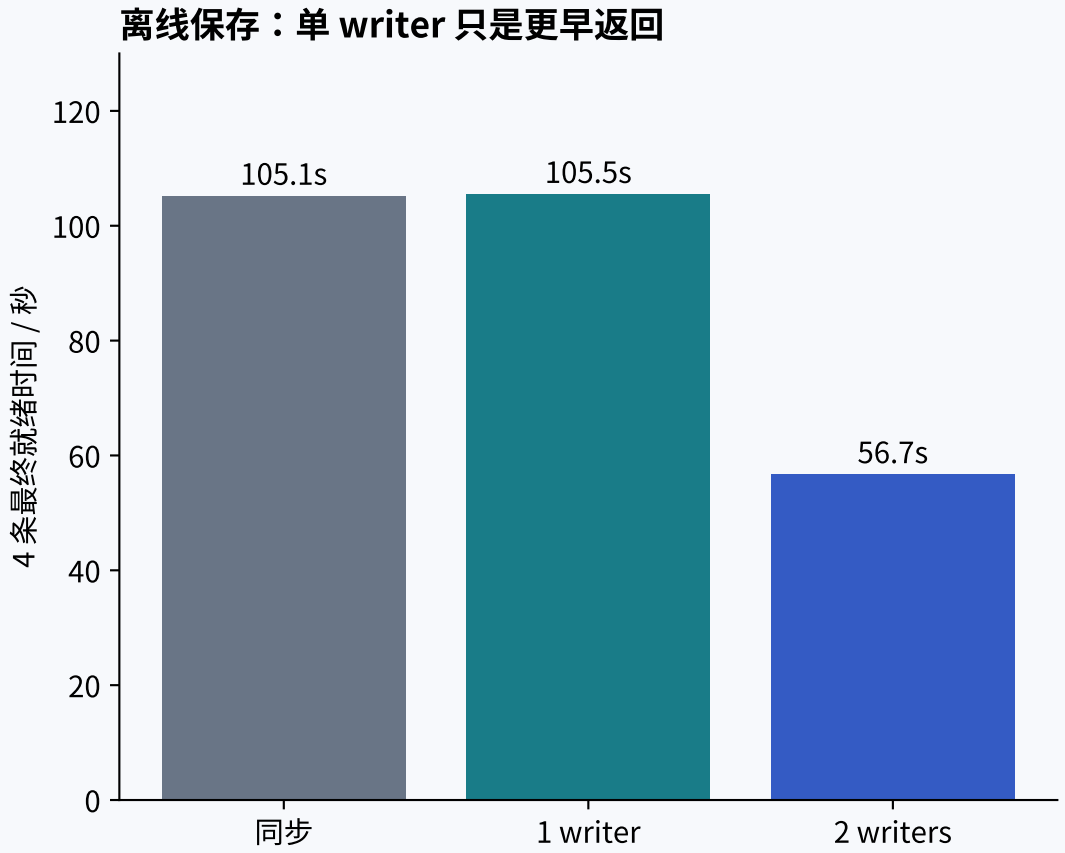
05 本轮提速未伴随已测末态通过率下降



当前实验：各组生成成功 16/17，保存数据末态检查通过 16/16。

支持结论：本轮已测末态通过率未下降；不代表全部质量指标或长期稳定性。



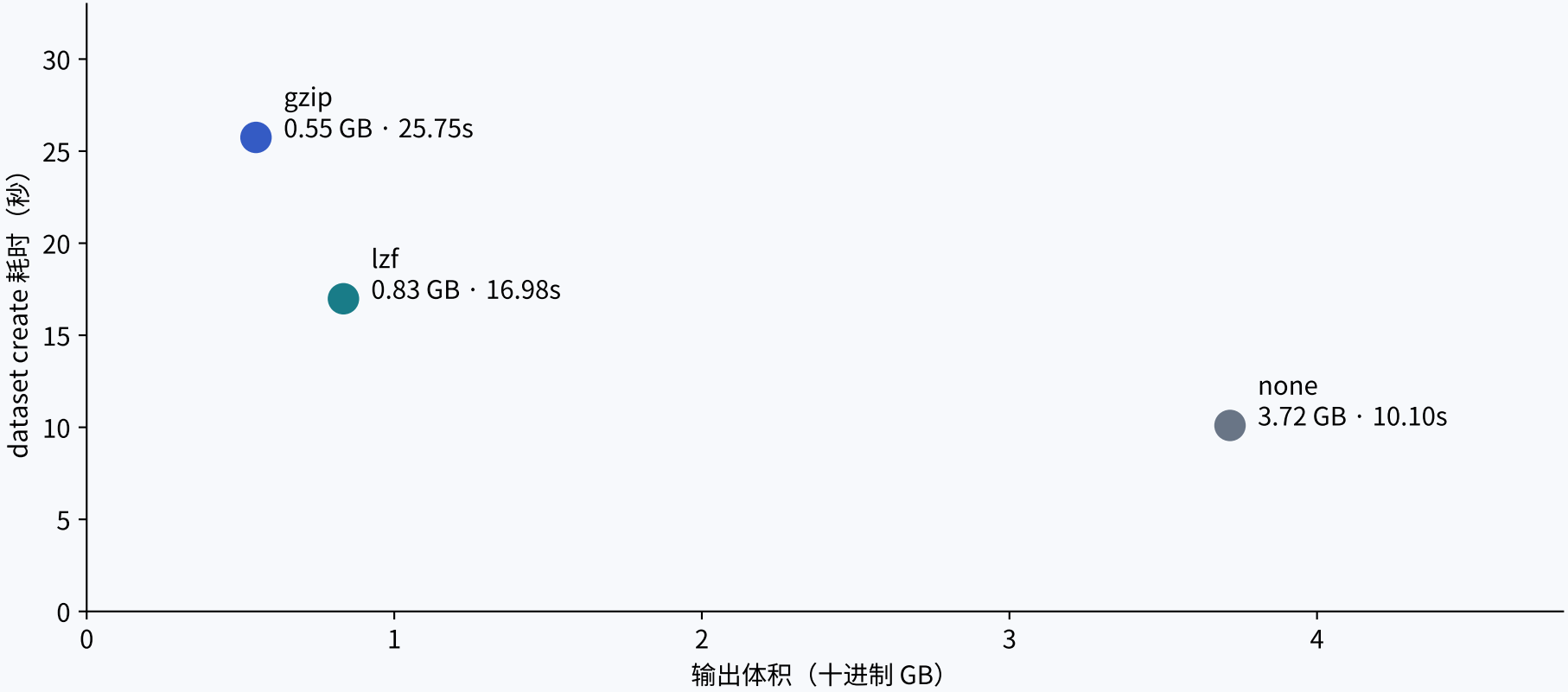


左：三个模式均为 24 条记录成功、23 条未态通过；右：CPU 离线回放，不能当作 GPU 生产吞吐。

当前实验：早期 24 条完整生产对照，以及 4 条数据的离线保存对照。

支持结论：双 writer 改善相应吞吐；离线单 writer 更早提交未缩短最终就绪时间。

07 压缩策略：速度与体积的取舍



1342 帧 / 相机的离线 RGB 回放，保持 chunk 可比，单轮；含压缩、HDF5 和 I/O，非完整生产。

当前实验：每相机 1342 帧 RGB 离线回放，固定 chunk，比较三种压缩方式。

支持结论：保存耗时与文件体积存在取舍；结果不能直接替代完整生产吞吐。

改进前：生成主流程等待导出



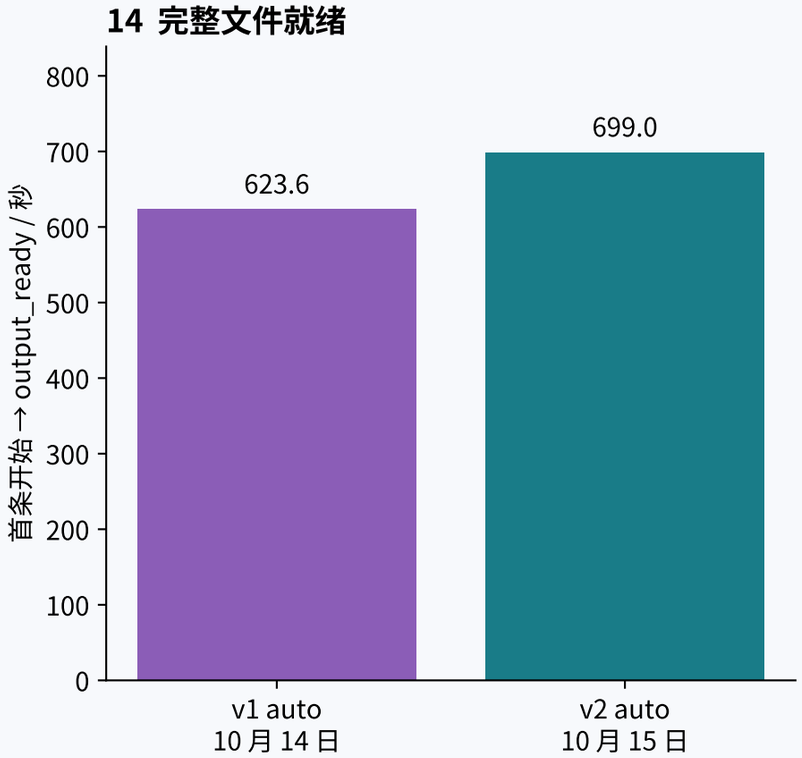
改进后：生成与保存重叠

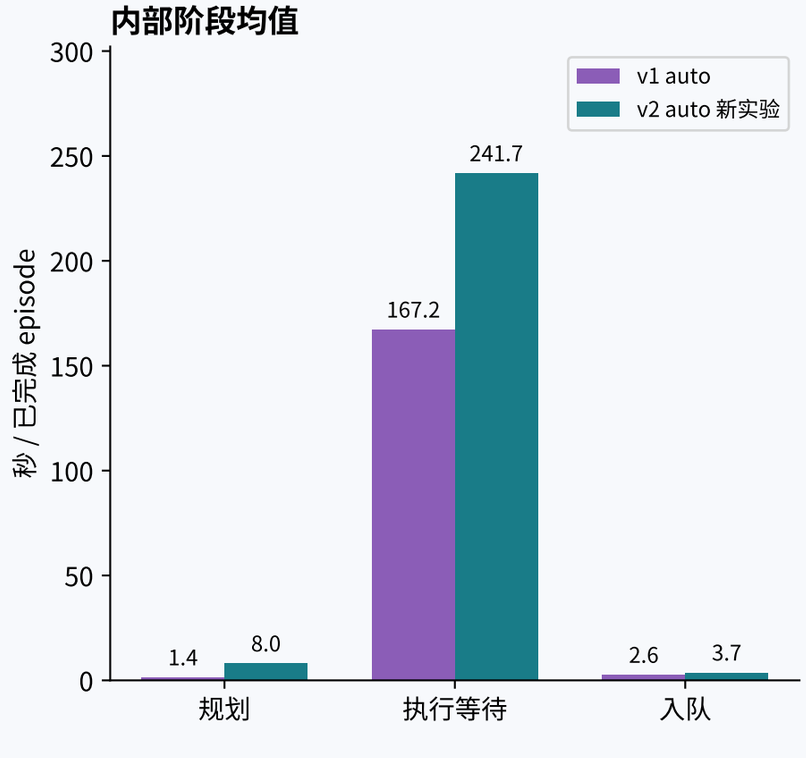


机制图，无时间比例。质量判定保持独立；异步提交完成不能代替最终文件就绪。

当前依据：同步导出与异步保存池的控制流；本页为机制示意。

支持结论：异步保存允许生成与保存重叠；图中箭头不表示实测时序或独立收益。





新实验检查（末态未全通过）

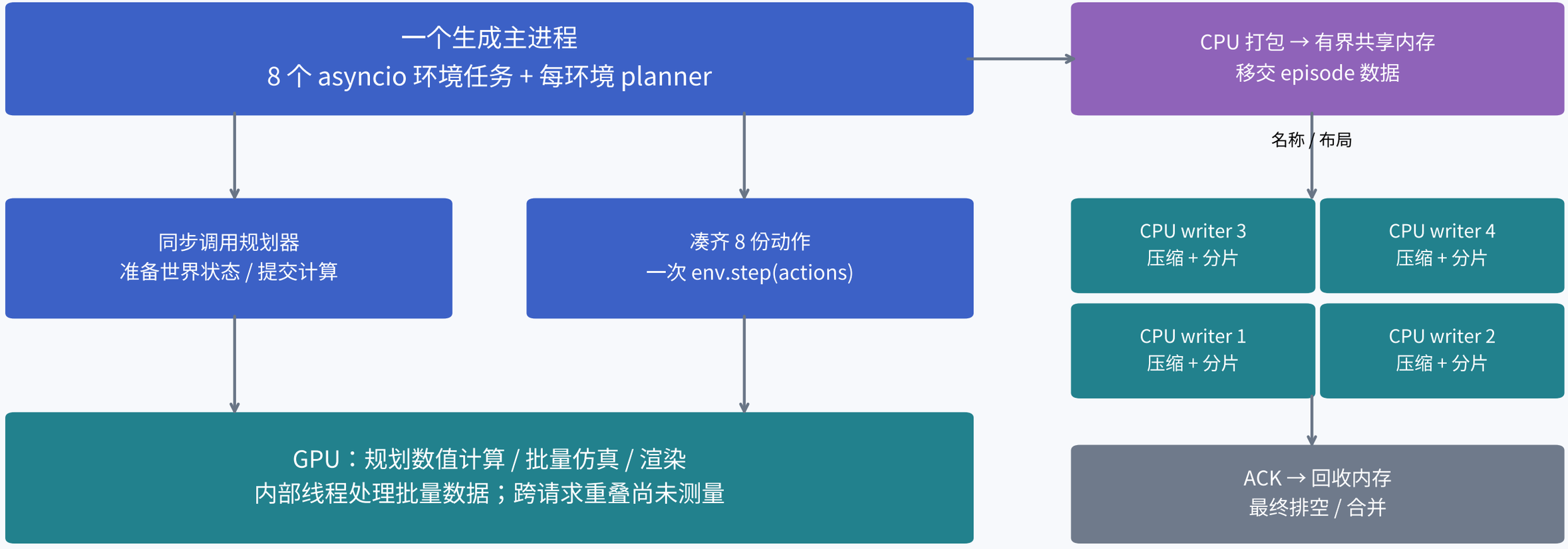
生成成功：16 / 16
最终保存：16 条
末态通过：15 / 16
实际 writer：4
记录成功：1.373 条 / 分钟
末态合格：1.287 条 / 分钟

跨日期单轮观测，缓存 / 热身 / 系统状态未控制一致；不能将差值单独归因于版本升级。

当前实验：v2 新重跑保存 16 条，最终就绪 699.05 秒，末态检查通过 15/16。

支持结论：补齐完整运行与输出校验，但质量未全通过；跨日期单轮不能隔离版本影响。

16 执行层次：协程并发、GPU 数据并行、CPU 多进程



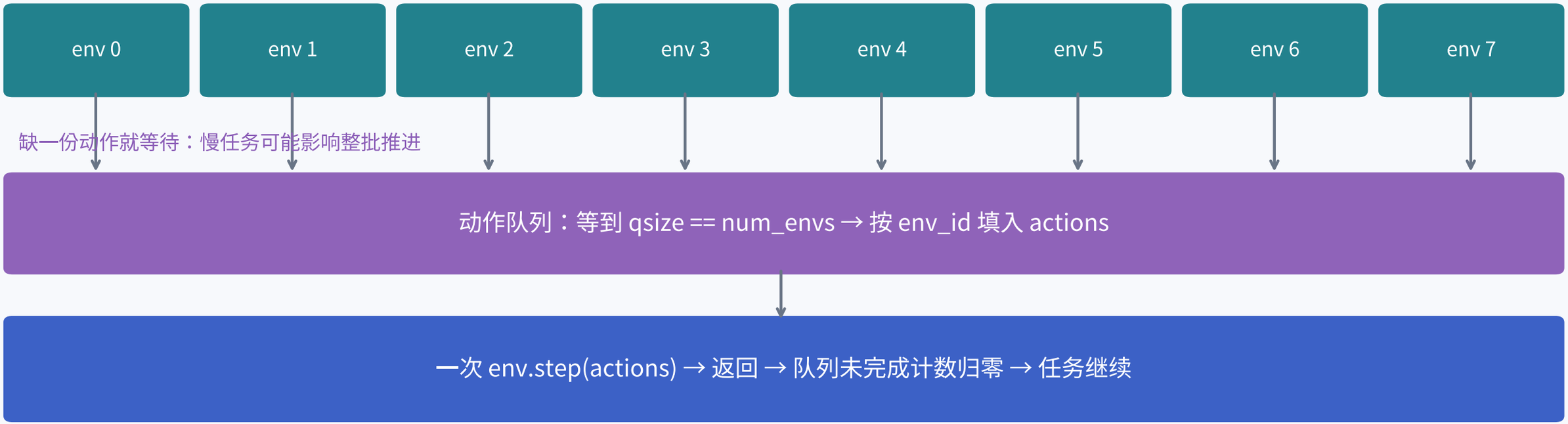
架构示意：8 个环境任务共享生成主进程；本轮异步组实际均为 4 个 CPU writer。

当前依据：主循环源码与保存池配置；本轮异步组实际启动 4 个 CPU writer。

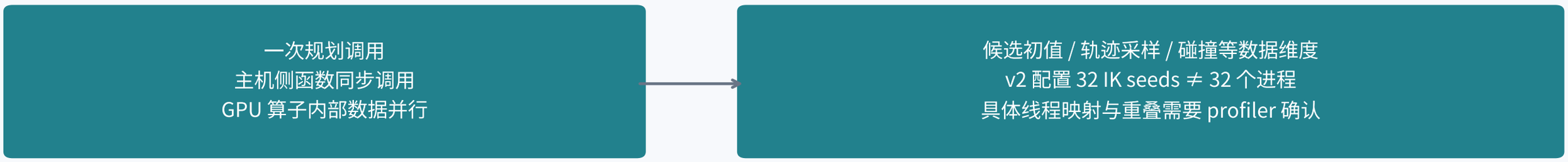
支持结论：环境任务、GPU 计算与 CPU 保存分属不同层次；未测跨请求 GPU 重叠。

17 环境在动作屏障汇合；GPU 算子内部执行数据并行

主进程：环境提交动作后等待 queue.join()；等待未完成时挂起协程



下方是规划计算原理示意，与上方动作屏障不是同一条执行时间线。



当前依据：动作队列汇合、env.step、task_done 与规划调用的代码路径。

支持结论：存在主机侧批次等待；队列完成记账不等于 CUDA 设备同步。

18 看并行单位与资源边界，不能只看数量

单进程多环境批量步进

当前生成侧

8 个环境任务协作调度
↓
动作汇合后批量步进
↓
GPU 数值计算

共享事件循环与仿真对象
同步调用 / 动作屏障会等待

CPU 多进程保存

当前保存侧

主进程打包 episode
↓ 共享内存
多个 writer 独立压缩
↓
各写分片 → 排空 / 合并

生成与保存可重叠
竞争 CPU、内存与磁盘

使用 GPU 的多个生产进程

未测试的另一种方案

进程 A：仿真 + 规划
进程 B：仿真 + 规划
↓
共享 GPU 或绑定不同 GPU
↓
协调任务与输出

需额外设备资源与调度
进程数不等于加速倍数

设备利用率不能证明 kernel 重叠；当前没有单 GPU 多进程共享或多 GPU 部署的吞吐对照。

当前依据：现有单进程批量生成与 CPU 多进程保存的实现对照。

支持结论：应按执行单位区分并发与并行；单 GPU 多进程和多 GPU 部署尚无对照实验。